

Azure Delegated SAS Token

Upload Architecture · .NET 10 API · Next.js Frontend · APIM

Azure

Ce document décrit l'architecture et le code nécessaires pour implémenter l'upload de fichiers vers Azure Blob Storage via un **Delegated SAS Token** (User Delegation Key), dans deux modes : depuis le **frontend Next.js** et depuis un **client externe via APIM**.

■ Stack: .NET 10 + Next.js 14+

■ Auth: Entra ID (Azure AD)

■ Storage: Azure Blob Storage

■ Gateway: Azure APIM

1. Qu'est-ce qu'un Delegated SAS Token ?

Un **Shared Access Signature délégué** est signé à l'aide d'une **User Delegation Key** obtenue auprès d'Azure Entra ID. Contrairement au SAS de compte (signé avec la clé de stockage), il repose sur les identités Entra ID et peut être révoqué sans rotation de clé.

Critère	SAS Token délégué	SAS Token de compte	SAS Token de service
Clé utilisée	User Delegation Key (Entra ID)	Clé de compte Storage	Clé de compte Storage
Durée max	7 jours	Illimitée	Illimitée
Révocable ?	✓ Oui (via Entra)	✗ Non (rotation clé)	✗ Non (rotation clé)
Recommandé prod.	✓ Oui	✗ Non	Non idéal
Managed Identity	✓ Requis	Optionnel	Optionnel
Permissions fines	✓ Oui	✓ Oui	✓ Oui

2. Mode 1 — Upload depuis le Frontend Next.js

Schéma des appels

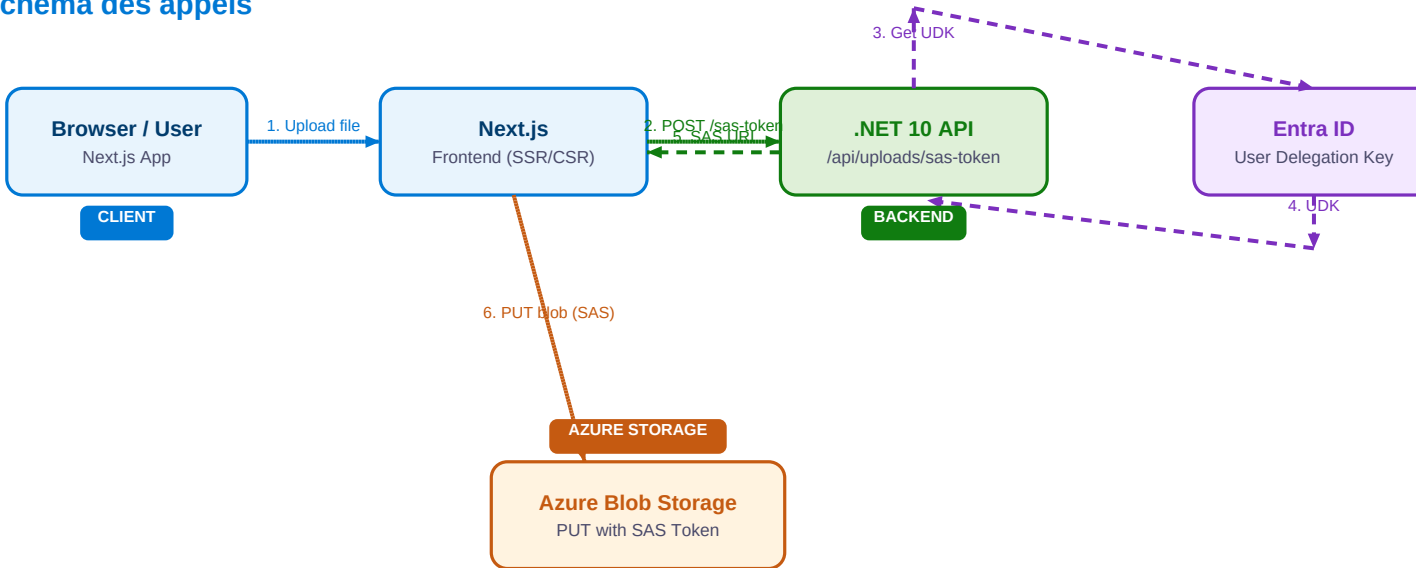


Fig. 1 — Flux d'upload depuis le browser via Next.js

Étapes détaillées

1

Sélection du fichier

L'utilisateur sélectionne un fichier dans le composant React. Le fichier reste en mémoire côté client.

2

Demande de SAS Token

Next.js appelle POST /api/uploads/sas-token avec le nom du fichier et le container cible. Un Bearer token Entra ID est joint.

3

Obtention de la User Delegation Key

L'API .NET 10 appelle GetUserDelegationKeyAsync() sur le BlobServiceClient. La clé est mise en cache jusqu'à 7j (renouvelée si expiration < 5 min).

4

Génération du SAS URI

L'API construit un BlobSasBuilder, signe avec la UDK et retourne un SAS URI valide 15 min avec permissions Write+Create.

5

Upload direct vers Azure

Le frontend exécute un PUT HTTP directement vers le SAS URI retourné. Aucun octet ne transite par l'API backend.

6

Confirmation

Azure retourne HTTP 201. Le frontend affiche la confirmation et l'URL publique (ou privée) du blob.

Code frontend — useFileUpload.ts (Next.js)

```

import { useState, useCallback } from 'react';

interface UploadOptions {
  containerName: string;
  folder?: string;
}

export function useFileUpload() {
  const [progress, setProgress] = useState<number>(0);
  const [uploading, setUploading] = useState(false);

  const upload = useCallback(async (
    file: File,
    { containerName, folder }: UploadOptions
  ) => {
    setUploading(true);

    // 1. Obtenir le SAS token depuis notre API
    const res = await fetch('/api/sas-token', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ containerName, folder, fileName: file.name }),
    });
    const { sasUri } = await res.json();

    // 2. Upload direct vers Azure Blob Storage
    await fetch(sasUri, {
      method: 'PUT',
      headers: { 'x-ms-blob-type': 'BlockBlob', 'Content-Type': file.type },
      body: file,
    });
    setUploading(false);
  }, []);

  return { upload, progress, uploading };
}

```

Route API Next.js — /app/api/sas-token/route.ts

```

import { NextRequest, NextResponse } from 'next/server';
import { getServerSession } from 'next-auth';

export async function POST(req: NextRequest) {
  const session = await getServerSession();
  if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const body = await req.json();

  // Proxy vers l'API .NET avec le token Entra ID de l'utilisateur
  const apiRes = await fetch(`${process.env.API_BASE_URL}/api/uploads/sas-token`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${session.accessToken}`,
    },
    body: JSON.stringify(body),
  });

  const data = await apiRes.json();
  return NextResponse.json(data);
}

```

Schéma de séquence

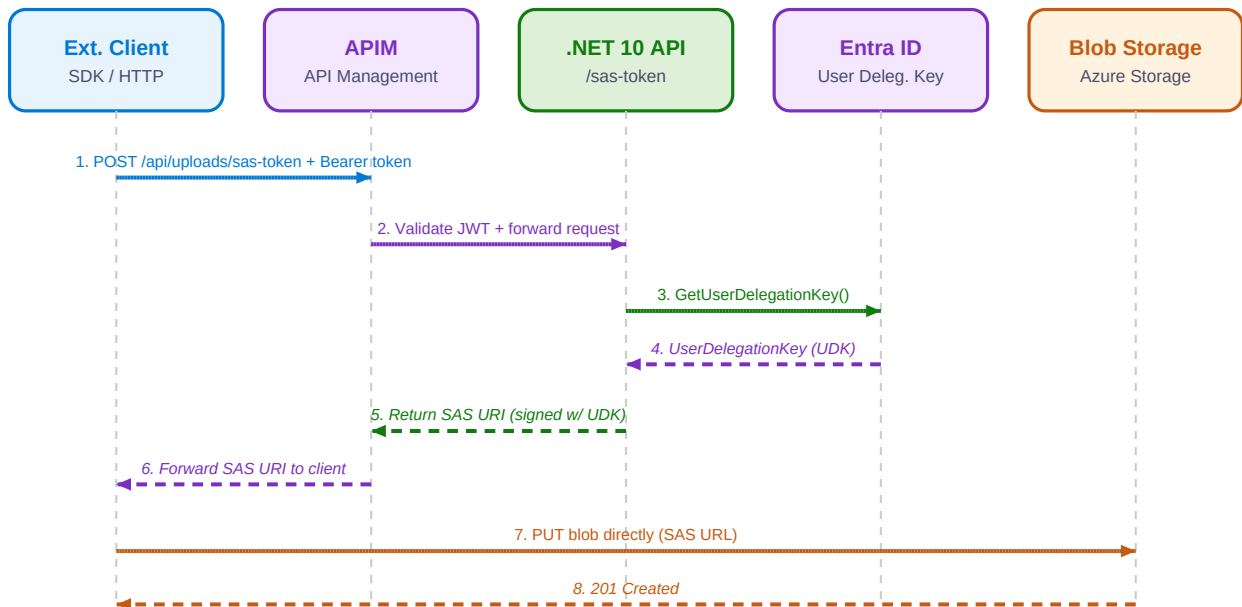


Fig. 2 — Flux d'upload depuis un client externe passant par APIM

Étapes détaillées

- 1 Appel client → APIM**
 Le client (SDK, service tiers, Postman...) envoie POST /api/uploads/sas-token avec un Bearer token Entra ID valide. La subscription APIM peut être requise (Ocp-Apim-Subscription-Key).
- 2 Validation JWT par APIM**
 La policy APIM validate-jwt vérifie le token, extrait les claims (roles, scopes) et rejette les appels non autorisés avant qu'ils atteignent le backend.
- 3 Forward vers .NET 10 API**
 APIM route la requête vers le backend en ajoutant des headers enrichis (x-ms-client-principal, correlation ID). Le SSL termination et rate limiting sont gérés ici.
- 4 User Delegation Key**
 L'API obtient la UDK via la Managed Identity de l'App Service. La clé est mise en cache (SemaphoreSlim thread-safe).
- 5 SAS URI retourné**
 L'API génère le SAS URI et le retourne via APIM au client. APIM peut appliquer des transformations de réponse (masquage, enrichissement).
- 6 Upload direct Azure**
 Le client effectue un PUT HTTP directement vers l'URL Azure Blob Storage avec le SAS. L'API backend n'est plus sollicitée.

Policy APIM — validate-jwt (inbound)

```

<!-- APIM Policy: inbound -->
<inbound>
<validate-jwt
header-name="Authorization"
failed-validation-httpcode="401"
failed-validation-error-message="Unauthorized">
<openid-config
url="https://login.microsoftonline.com/{tenantId}/v2.0/.well-known/openid-configuration" />
<audiences>
<audience>api://{YOUR_API_CLIENT_ID}</audience>
</audiences>
<required-claims>
<claim name="roles" match="any">
<value>Upload.Write</value>
</claim>
</required-claims>
</validate-jwt>
<set-header name="x-correlation-id" exists-action="skip">
<value>@(Guid.NewGuid().ToString())</value>
</set-header>
</inbound>

```

Exemple d'appel client (C# SDK)

```

using Azure.Identity;
using Azure.Storage.Blobs;

// 1. Obtenir un token Entra ID pour l'API
var credential = new ClientSecretCredential(tenantId, clientId, clientSecret);
var token = await credential.GetTokenAsync(
    new TokenRequestContext(new[] { $"api://{apiClientId}/.default" }));

// 2. Appeler APIM pour obtenir le SAS URI
var http = new HttpClient();
http.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", token.Token);
http.DefaultRequestHeaders.Add("Ocp-Apim-Subscription-Key", subscriptionKey);

var response = await http.PostAsJsonAsync(
    $"{apimBaseUrl}/api/uploads/sas-token",
    new { containerName = "documents", fileName = "report.pdf" });

var result = await response.Content.ReadFromJsonAsync<SasResponse>();

// 3. Upload direct (sans passer par APIM)
var blobClient = new BlobClient(new Uri(result!.SasUri));
await blobClient.UploadAsync(fileStream, overwrite: true);

```

4. Backend .NET 10 — BlobStorageService

Le service utilise **DefaultAzureCredential** pour s'authentifier auprès d'Azure (Managed Identity en production, az login en local). La **User Delegation Key** est mise en cache (thread-safe via SemaphoreSlim) pour éviter des appels répétés à Entra ID.

```
public async Task<DelegatedSasTokenResponse> GetDelegatedSasTokenAsync(
    string containerName, string blobName,
    BlobSasPermissions permissions, TimeSpan validity)
{
    var expiresOn = DateTimeOffset.UtcNow.Add(validity);

    // 1. Obtenir / réutiliser la User Delegation Key
    var udk = await GetOrRefreshUserDelegationKeyAsync(expiresOn);

    // 2. Construire le SAS Builder
    var sasBuilder = new BlobSasBuilder {
        BlobContainerName = containerName,
        BlobName = blobName,
        Resource = "b", // blob-level SAS
        ExpiresOn = expiresOn,
    };
    sasBuilder.SetPermissions(permissions); // Write | Create

    // 3. Signer avec la UDK (pas de clé de compte !)
    var sasParams = sasBuilder.ToSasQueryParameters(u dk, _accountName);

    return new DelegatedSasTokenResponse(
        SasUri: $"https://{_accountName}.blob.core.windows.net/{containerName}/{blobName}?{sasParams}",
        ExpiresOn: expiresOn, ...
    );
}
```

5. Configuration IAM & RBAC Azure

Ressource	Identité	Rôle RBAC requis	Pourquoi
Azure Blob Storage	Managed Identity (App Service / AKS)	Storage Blob Data Contributor	Générer la UDK et accéder aux blobs
Azure Blob Storage	Développeur local (az login)	Storage Blob Delegator + Data Contributor	Tester sans clé de compte
Azure AD App (API registration)	Client APIM / Frontend	Role: Upload.Write (App Role défini)	Contrôle d'accès à l'endpoint SAS

6. Bonnes pratiques & Sécurité

■ À faire

- Utiliser DefaultAzureCredential (pas de clé en dur)
- Durée SAS courte : 15 min pour upload interactif
- Valider le nom du fichier (path traversal)
- Mettre en cache la User Delegation Key (max 7j)
- Restreindre le SAS à un blob précis (Resource="b")
- Activer Azure Defender for Storage
- Logs : tracer qui génère quel SAS (Correlation ID)

■ À éviter

- Ne jamais stocker la clé de compte dans le code
- Ne pas exposer le SAS URI dans les logs applicatifs
- Éviter les SAS de conteneur (trop permissifs)
- Ne pas désactiver la validation JWT dans APIM
- Éviter les durées > 1h en mode production
- Ne pas bypasser APIM pour les clients externes
- Ne pas utiliser SAS anonyme sans authentification

7. Variables d'environnement & Configuration

```
# appsettings.json (API .NET 10)
{
  "AzureAd": {
    "TenantId": "<TENANT_ID>",
    "ClientId": "<API_CLIENT_ID>"
  },
  "Azure": { "Storage": { "AccountName": "<STORAGE_ACCOUNT>" } },
  "AllowedOrigins": ["https://your-app.vercel.app"]
}

# .env.local (Next.js)
API_BASE_URL=https://your-api.azurewebsites.net
NEXTAUTH_SECRET=<secret>
AZURE_AD_CLIENT_ID=<frontend_client_id>
AZURE_AD_TENANT_ID=<tenant_id>
AZURE_AD_CLIENT_SECRET=<client_secret>
```